

Lecture 7 — Live Forensics

F20FO / F21FO — Digital Forensics 2025–26

Final Exam–Ready Notes

Contents

1	What Is Live Forensics? (The Big Picture)	3
1.1	Live vs. Dead/Static Forensics	3
2	Why Do We Need Live Forensics?	3
2.1	The Need	3
2.2	The Opportunity	3
3	Live Forensics Goals	4
4	Data Lifetime & Volatile Data	4
4.1	Why Volatile Data Matters	4
4.2	Persistence of Memory (Post-Reboot)	5
5	Constraints & Rules of Live Forensics	6
5.1	Key Constraints	6
5.2	Minimising Your Footprint	6
6	Dangers of Live Forensics — Rootkits	6
6.1	Two Types of Rootkits	7
6.2	How Kernel-Mode Rootkits Fool Everything	7
7	Approach to Live Forensics	8
7.1	Evidence Acquisition Scenarios	8
7.2	Accessing the System	8
8	Live Forensics Information Sources	9
8.1	What You Can Collect	9
8.2	Running Processes — What You Can Learn	9
8.3	Memory Sources	10
9	Live Forensics Analysis	10
10	Live Forensics Tools & Commands	11
10.1	Windows — Sysinternals Suite	11
10.2	Additional Windows Tools (vidstromlabs)	11
10.3	Windows Network Tool — Netstat	11
10.4	Linux — Process Commands	12
11	Software-Based Memory Acquisition	13
11.1	Using the dd Command	13
11.1.1	Linux Physical Memory Sources	13
11.2	Other Software-Based Methods	13
11.3	Problems with Memory Acquisition	13
12	Step-by-Step Walkthrough: Reading Process Memory	14
12.1	Background: Process Memory Segments	14
12.2	The Walkthrough	14

13 Live Disk Imaging & File Carving	16
13.1 Live Disk Imaging	16
13.2 Live File Carving	16
14 Virtual Machine Forensics	16
15 Lab 7: Hashing, Forensic Imaging & Data Analysis	17
15.1 Lab 7.1: Hashing	17
15.1.1 Why Hashing Matters for Forensics	17
15.1.2 How to Create Hashes	18
15.1.3 Lab 7.1.1 — Hashing Exercise: Identifying Identical Files	18
15.1.4 Lab 7.1.2 — Hashing Exercise: Seeing the Changes	18
15.2 Forensic Use of Hashing	19
15.3 Lab 7.2: Digital Forensic Imaging and Analysis	19
15.3.1 Lab 7.2.1 — Autopsy: Setup and Case Creation	20
15.3.2 The Murder Case Scenario	20
15.3.3 Autopsy Case Creation — Step-by-Step Procedure	21
15.3.4 Autopsy Investigation Questions	22
15.3.5 Lab 7.2.2 — FTK (Forensic Toolkit)	22
15.3.6 FTK Investigation Questions	23
15.4 MAC Times	23
15.4.1 MAC Times Lab Question	23
16 Lab 8: Live Forensics and Incident Response	24
16.1 Lab 8.1: VM and Autopsy	24
16.1.1 Why VMs Matter for Forensics	24
16.1.2 Lab 8.1 Procedure: Setting Up a VM for Forensic Analysis	24
16.2 Lab 8.2: Procmon (Process Monitor)	25
16.2.1 Key Procmon Controls	25
16.2.2 Lab 8.2.1 — Procmon Exercise: Tracing a Registry Change	25
16.3 Lab 8.3: Live Forensics	26
16.3.1 Static vs. Dynamic Linking — Why It Matters	27
16.3.2 Lab 8.3.1 — Setting Up the FIRE Disk	27
16.3.3 Lab 8.3.2 — Collecting Live State Manually	28
16.3.4 Key Analysis: Comparing Process Lists	28
16.3.5 Creating Memory Images	29
16.3.6 Lab 8.3.3 — Automated IR Scripts	29
16.3.7 Lab 8.3.4 — Checking for Rootkits with chkrootkit	29
16.3.8 Lab 8.3.5 — Offline Analysis of RAM Dump	30
16.3.9 Lab 8.3.6 — Windows Incident Response	30
16.3.10 Shutdown Considerations	30
17 Complete Process Summary Flowchart	31
18 Quick-Reference Exam Cheat Sheet	33

1 What Is Live Forensics? (The Big Picture)

Definition — Live Forensics

Live forensics is the process of collecting digital evidence from a computer (or other device) *while it is still turned on and running*.

Think of it like this: Imagine police arrive at a crime scene and the suspect's laptop is open on the desk, programs running, websites visible. Live forensics means carefully gathering information from that running laptop *before* you power it off.

1.1 Live vs. Dead/Static Forensics

Feature	Live Forensics	Dead / Static Forensics
System state	Computer is ON and running	Computer is OFF ; work from a disk image
Data type	Volatile data (RAM, running programs, open network connections)	Non-volatile data (files saved on the hard drive)
Risk	You might accidentally <i>change</i> something on the running system	Evidence is “frozen” — less risk of change
Unique advantage	Can capture passwords, encryption keys, chat sessions, live network traffic	Can search entire disk without time pressure
Real-world analogy	Photographing a whiteboard before someone erases it	Reading a printed letter after the meeting is over

The exam loves asking you to *compare* live and dead forensics. Memorise the table above — especially the “volatile vs. non-volatile” distinction.

2 Why Do We Need Live Forensics?

There are two sides: **Need** (we have no choice) and **Opportunity** (it gives us extra evidence).

2.1 The Need

1. **Huge storage disks.** Modern servers have terabytes of data. Copying (“mirroring”) an entire 4TB disk takes many hours. You may not have that time.
2. **Mission-critical systems.** Suppose the suspect's computer is a hospital server running life-support monitoring. You cannot just pull the plug — people could die. You *must* collect evidence while it runs.
3. **Court orders are limited.** A warrant might only let you look at *one* specific server out of 30. You need a quick, live check to justify a broader warrant.

2.2 The Opportunity

1. **Speed.** A quick live scan can be done in minutes — no need to haul away hardware.

2. **Exclusive data.** Some evidence *only* exists in RAM (temporary memory). Once you power off, it is **gone forever**. Examples:
 - Passwords typed into a web browser
 - Encryption keys protecting a locked hard drive
 - Chat messages that were never saved to disk
3. **Context.** Seeing what is running *right now* helps you decide which parts of the hard drive to focus on later during static analysis.

Example:

Why RAM matters Imagine a suspect is logged into their online banking with a password manager. The password exists in the computer's RAM right now. If you turn the computer off first, the password disappears from RAM and you must crack the encrypted password file instead — which could take weeks or be impossible.

3 Live Forensics Goals

When you perform live forensics, you are trying to answer these questions:

Goal	Plain-English Explanation
What is happening now?	Which programs are running? What websites are open?
Who is doing what?	Which user account is active? Are there remote connections?
Get another puzzle piece	Combine live data with hard-drive data to build the full picture
Focus static analysis	Narrow down which folders or files to examine later
Passwords & keys	Grab passwords, decryption keys, unencrypted documents from RAM
Reconstruct sessions	Rebuild web browsing sessions, FTP file transfers, chat conversations
Find files	Spot downloaded files or files accessed from network drives
Identify remote machines	See which other computers the suspect was connected to

4 Data Lifetime & Volatile Data

Definition — Volatile Data

Volatile data = information stored in RAM (temporary memory) that **disappears when power is removed**.

Non-volatile data = information stored on disk (hard drive, SSD, USB) that **survives a power-off**.

4.1 Why Volatile Data Matters

- Most applications are *not* designed to erase sensitive data from RAM. A password you type may get **duplicated dozens of times** across different memory buffers.

- Even though data is often encrypted *on disk*, it is usually **not encrypted in RAM** (otherwise the CPU could not process it).
- Places a password might sit in RAM:
 - Kernel buffers (the operating system’s internal storage)
 - Window manager memory (the part that draws your screen)
 - Application buffers (the browser’s own storage)
 - Dynamic memory allocator (general-purpose memory pool)

Example:

The Password Duplication Problem You type your email password into Firefox. Here is what happens in RAM:

1. The keyboard driver stores each keystroke → **copy 1**
2. The window manager passes it to Firefox → **copy 2**
3. Firefox stores it in a text field buffer → **copy 3**
4. Firefox sends it to the network module → **copy 4**
5. The OS kernel buffers the network packet → **copy 5**

That is at least 5 copies of your password floating around in RAM, even though you only typed it once.

4.2 Persistence of Memory (Post-Reboot)

- Some systems *retain* data in RAM even after a reboot (warm reboot or even sometimes a cold reboot).
- This depends on the hardware model and BIOS settings.
- This can be used as a “last resort” to recover live data if the computer was recently switched off.
- This is an active research area in **memory forensics**.

Be ready to explain that RAM data can sometimes survive a reboot, making memory forensics possible even after shutdown.

5 Constraints & Rules of Live Forensics

5.1 Key Constraints

#	Constraint
1	You need physical or remote access to the running system (even if only for a short time).
2	Minimise changes. Everything you do on the system (inserting a USB, running a tool) <i>changes</i> something. You must keep changes as small as possible. Remember ACPO Principle 1: no action should change data which may be relied upon in court.
3	Minimise impact on the system, especially if it is mission-critical (e.g. a hospital server).
4	Record every step. Write down every command you run and every USB you plug in. If you leave “fingerprints,” you must be able to explain them in court.

Example:

The Detective’s Fingerprints Think of a real crime scene. A detective picks up a ransom note with bare hands and leaves fingerprints on it. Now the defence lawyer asks: “How do we know those aren’t the criminal’s fingerprints?”

The same thing happens in digital forensics. If you run a program on the suspect’s computer, you leave traces (file dates change, memory shifts). You *must* document exactly what you did so no one can argue you planted evidence.

5.2 Minimising Your Footprint

- Use **trusted, pre-compiled tools** — ideally carry them on a read-only USB so you do not install anything on the suspect’s machine.
- Keep a **low profile** — especially if the suspect might be monitoring the system (you do not want to tip them off).
- Any artifacts (changes) you create **can be explained in court** if properly documented.

6 Dangers of Live Forensics — Rootkits

Definition — Rootkit

A **rootkit** is a type of malware that hides itself (and other malware) from the operating system and from forensic tools. It is like an invisible burglar who also makes the security cameras show a fake “all clear” feed.

6.1 Two Types of Rootkits

	User-Mode Rootkit	Kernel-Mode Rootkit
Difficulty	Shallow trickery (easier to make)	Deep trickery (very advanced)
How it works	Replaces normal commands like ls (list files), ps (list processes), du/df (disk usage) with fake versions that hide the malware	Modifies the OS kernel itself (the core brain of the operating system) to produce fake results
What it hides	Changes disk space statistics, hides processes from the process list	Can hide files, blocks of memory, entire programs — even if you bring your own trusted tools
Defence	Bring your own trusted copies of ls , ps , etc. on a USB	Very hard to detect; even trusted tools can be fooled because the OS kernel is compromised

6.2 How Kernel-Mode Rootkits Fool Everything

- **Disk drivers:** The rootkit intercepts all disk reads, so it can hide files at the hardware level.
- **Virtual memory system:** It hacks the memory management, hiding malicious processes.
- **Shared libraries (DLLs):** It replaces shared code libraries. Even your “trusted” executable uses DLLs from the OS — if those are replaced, your trusted tool is compromised.
- The *only* true defence is to use **statically linked executables** (programs that contain all their code inside themselves and do not rely on the OS for shared libraries).

Exam Favourite The exam may ask: “Why might your trusted forensic tools still give wrong results on a live system?” Answer: **Kernel-mode rootkits** modify the OS itself. Even a clean copy of **ls** or **ps** relies on the OS kernel to read files and processes. If the kernel is compromised, it feeds lies to every tool. The only protection is statically linked binaries.

7 Approach to Live Forensics

7.1 Evidence Acquisition Scenarios

Scenario	What You Do
Triage	You have <i>limited time</i> . Quickly answer: Is there a problem? Which machines are affected? What disks need imaging? Focus on where the problem is worst and evidence is most abundant.
Illegal Server	Compare what services are running on the machine vs. what a network scan (e.g. nmap) sees from outside. Are there hidden services? Some techniques are subtle — for example port knocking (a hidden sequence of connection attempts that “unlocks” a secret port), but most are obvious. Check network traces to see who is talking to the computer. Be careful of legal issues when examining network traces — you may need separate authorisation to intercept or monitor network traffic.
Trojan Defence	Suspect claims: “It wasn’t me, a Trojan/hacker controlled my computer!” You must support or refute this by checking: • Are there traces of known Trojans? • Are there unusual network connections? • What is running on the machine? • Is anything hidden? • Is there enough other evidence to corroborate? First used in UK court in 2003. Still tricky because malware is often present on a system but may not actually be relevant to the case — just because a Trojan exists does not automatically mean it caused the crime. You must determine whether the specific malware found could have produced the observed behaviour.

7.2 Accessing the System

When	How
Before investigation	Use pre-installed agents (monitoring software set up in advance). The agents must not have been compromised.
During investigation	Need login credentials . Must use known-good binaries (trusted tools). All artifacts (changes you make) must be accounted for .

8 Live Forensics Information Sources

8.1 What You Can Collect

Source	What It Contains
Entire disk image	A full copy of the hard drive taken while the system is running (“live disk imaging”). Problem: files are constantly changing, so the image is a moving target.
Deleted files	Recovered via “live file carving” — scanning the raw disk for fragments of deleted files using tools like Scalpel or Foremost.
Unencrypted fragments	Pieces of documents that are temporarily stored in plain text in RAM.
Encryption keys	Keys used for full-disk encryption (e.g. BitLocker) may be sitting in RAM. Capture them and you can later decrypt the entire drive.
Volatile malware	Some malware <i>only</i> lives in RAM and never writes to disk. If you power off, the malware is gone and you have no evidence.

8.2 Running Processes — What You Can Learn

Information	Windows	Linux / Unix
Open files		
Open network connections		
Registry activity		— (Linux has no registry)
Open DLLs / shared libs	(DLLs)	(shared libraries)
Access to deleted EXE	—	(can still access the executable even if the file was deleted from disk)
Command line used	—	
Environment variables	—	

8.3 Memory Sources

Memory Type	Details
Process memory	Most useful. Each program has its own virtual memory space. Easier to search than dumping all of RAM because the data is organised. Can yield passwords, document fragments, decryption keys.
Kernel memory	Used to search for “hidden” processes (ones a rootkit might be hiding). Also helps evaluate the health of the kernel (is it compromised?).
String searches	The “brute force” approach: search the whole of RAM for text strings like “password”, credit card numbers, email addresses, etc.

9 Live Forensics Analysis

Once you have collected live data, you need to **analyse** it. The lecture identifies several specific analysis questions:

Analysis Question	What You Are Looking For
Are things “not right”?	Are processes or files hidden ? A rootkit may be concealing malware from the process list or hiding files from directory listings. Compare outputs from multiple tools.
Is disk encryption in use?	If full-disk encryption (e.g. BitLocker, LUKS) is active, the decryption key is in RAM <i>right now</i> . You must capture it before power-off or you may never access the disk contents.
What is staying in memory?	A lot of data persists in RAM longer than expected. Dumping process memory can reveal passwords, documents, chat logs, and session tokens that were never deliberately saved to disk.
Process memory dumps	For each suspicious process, dump its entire memory space and analyse it offline for strings, structures, keys, etc.
Dumping OS structures	The operating system maintains internal data structures that track processes and memory. By dumping these, you can determine which areas of swap space correspond to a particular process — meaning you can link specific chunks of virtual memory on disk back to the application that owned them.

The lecture treats “Analysis” as a *separate step* from “Information Sources.” The key distinction is: information sources tell you *where* to look; analysis tells you *what questions to ask* once you are looking. Remember the swap-space mapping point — it is a specific detail the exam may test.

10 Live Forensics Tools & Commands

10.1 Windows — Sysinternals Suite

Sysinternals is a collection of free tools by Microsoft for examining Windows systems:

Tool	What It Does (Plain English)
FileMon	Shows file system activity in real time (which files are being read/written right now)
PSMon	Watches processes and threads being created in real time
PsFile	Shows files that are being opened by remote users (someone accessing files over the network)
PsKill	Kill (stop) a process by name or process ID
PsInfo	Lists general system information
PsList	Lists detailed information about all running processes
PsLoggedOn	Shows who is currently logged in (both locally and over the network)
PsLogList	Dumps event log records (system logs showing what happened)
PsPasswd	Changes account passwords (useful for gaining access during investigation)
PsService	View and control Windows services
PsSuspend	Pause (suspend) a running process without killing it
Handle	Shows which files are opened by which processes
RegMon	Shows registry activity in real time
ListDLLs	Shows which DLLs (shared code libraries) are loaded by each process

10.2 Additional Windows Tools (vidstromlabs)

Tool	What It Does
Pmdump	Dumps a specific process's memory (given its PID)
ListDrivers	Lists all loaded kernel drivers
ListModules	Lists EXEs and DLLs associated with a process
PromiscDetect	Detects if a network adaptor is in promiscuous mode (meaning it is capturing <i>all</i> network traffic, not just its own — a sign someone might be sniffing the network)

10.3 Windows Network Tool — Netstat

netstat — View Network Connections

```
netstat -a          Shows ALL active network connections
netstat -a -b       Same, but also shows which EXE owns each connection
```

Example:

Using netstat You run `netstat -a -b` on the suspect's Windows machine and see:

```
TCP    192.168.1.5:443    suspicious-server.ru:8080    ESTABLISHED    [
    malware.exe]
```

This tells you:

- A program called `malware.exe` is running
- It has an active connection to a server in Russia on port 8080
- This is strong evidence of compromise

10.4 Linux — Process Commands

`ps` — List Running Processes

```
ps aux | less
```

Breaking it down:

- `a` = show processes for **all users** (not just yours)
- `u` = show the **user/owner** of each process
- `x` = also show processes **not attached to a terminal** (background services)
- `| less` = pipe the output through a pager so you can scroll through it

The output looks like this:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
mj	2963	0.0	0.0	4688	1284	pts/0	T	11:21	0:00	python3
mj	3107	0.0	0.0	9152	2392	pts/1	S+	11:28	0:00	grep python3

Column	Meaning
USER	Who owns the process
PID	Process ID — a unique number identifying this process
%CPU	How much CPU time the process is using
%MEM	How much RAM the process is using
VSZ	Virtual memory size (total memory the process <i>can</i> use)
RSS	Resident Set Size (how much RAM it is <i>actually</i> using right now)
TTY	Which terminal the process is running on (e.g. pts/0)
STAT	Process state (T = stopped, S = sleeping, R = running)
START	When the process started
TIME	Total CPU time consumed
COMMAND	The command that launched this process

11 Software-Based Memory Acquisition

11.1 Using the dd Command

Definition — dd

dd is a Unix/Linux command-line tool that copies raw data block by block. Think of it as a “photocopy machine for data” — it can copy files, disks, or even sections of RAM bit-for-bit.

11.1.1 Linux Physical Memory Sources

Path	What It Is
/dev/mem	Direct access to physical RAM (problematic on modern systems — restricted for security)
/proc/kcore	Kernel virtual memory — a view of what the kernel sees in RAM
/proc/[PID]/mem	Memory for a specific process (replace [PID] with the process number)

Key Problem dd relies on the operating system to read memory. If the OS is compromised (e.g. by a rootkit), it might give you *fake* data!

11.2 Other Software-Based Methods

- **Virtual machines (VMware, VirtualBox):** You can *freeze* a VM, take a snapshot of its entire disk and memory, and then resume it. You can also dump memory over the local network connection between host and VM.
- **Hibernation files:** When a computer hibernates, it writes a snapshot of physical memory to disk. This file can be analysed later like a memory dump.

11.3 Problems with Memory Acquisition

Problem	Explanation
Software tools overwrite evidence	Loading any tool into memory <i>erases</i> some existing data in RAM. You cannot avoid this entirely.
Trust issue	Software tools must ask the OS for memory access. If the OS is compromised, it can lie.
Hardware solutions	Hardware memory capture cards exist but must be pre-installed before the investigation.
Hardware poisoning	Sophisticated attackers can trick hardware-based capture too.

12 Step-by-Step Walkthrough: Reading Process Memory

12.1 Background: Process Memory Segments

Before diving into the walkthrough, you need to understand how a running program's memory is organised. The lecture (and Lecture Topic 3) explains that each **process** (running application) gets its own block of memory, divided into **segments**:

Segment	What It Stores	Real-World Analogy
Text	The machine code of the compiled program — the actual CPU instructions. This segment is usually <i>read-only</i> .	The printed recipe in a cookbook — you read it but don't write on it.
Stack	Manages the function/method call structure during execution. Every time a function is called, a new "frame" is pushed onto the stack; when it returns, the frame is popped off. Local variables live here.	A stack of sticky notes — you add one on top each time you start a sub-task, and remove it when you finish.
Heap	Contains dynamically allocated memory — memory that the program explicitly requests at runtime (e.g. when you create a new object or variable). This is where data like our "Hello World!" string ends up.	A whiteboard where you write things as needed — you decide at runtime what goes there.

The lecture specifically introduces text, stack, and heap *before* the Python walkthrough. If the exam asks "What are the memory segments of a process?" you must name all three and explain each.

12.2 The Walkthrough

This is a **key scenario from the lecture**. Follow each step carefully.

Scenario: Finding "Hello World!" in a Running Python Program's Memory

Goal: A Python program has a variable `hello = "Hello World!"`. We want to *prove* this string exists in the program's memory by reading raw RAM.

Step 1: Start Python and create the variable

Open a terminal and start Python:

```
$ python3
>>> hello = "Hello World!"
```

Step 2: Find the memory address of the variable

Python can tell us where in memory this variable lives:

```
>>> id(hello)
129592570691632
```

This number (129592570691632) is the **memory address** — like a street address for where "Hello World!" is stored in RAM.

Step 3: Find the Process ID (PID)

Each running program has a unique PID. Since Python is running in our current terminal, we need to open a *second* terminal and search for it:

```
$ ps aux | grep python3
USER      PID    %CPU %MEM    VSZ   RSS  TTY    STAT  START  TIME  COMMAND
mj        2963     0.0  0.0   4688   1284 pts/0    T     11:21  0:00  python3
```

Our Python process has PID = **2963**.

Step 4: Read the memory using dd

```
$ sudo dd bs=1 skip="129592570691632" count=1024 \
  if="/proc/2963/mem" status=none | hexdump -C
```

Breaking down this command:

- **sudo** — run as administrator (needed to read another process's memory)
- **dd** — the data copying tool
- **bs=1** — read **1 byte at a time** (block size = 1)
- **skip="129592570691632"** — **jump to this memory address** (skip this many bytes from the start)
- **count=1024** — read **1024 bytes** starting from that address
- **if="/proc/2963/mem"** — input file is the memory of process 2963
- **status=none** — suppress progress output
- **| hexdump -C** — display the raw bytes in hexadecimal + ASCII

Step 5: Read the output

```
00000000  01 00 00 00 00 00 00 00  c0 62 a4 00 00 00 00  |.....b
.....|
00000010  0c 00 00 00 00 00 00 00  af 43 bb da 85 84 d6 24  |.....C
.....$|
00000020  64 00 00 00 00 00 00 00  48 65 6c 6c 6f 20 57 6f  |d.....
Hello Wo|
00000030  72 6c 64 21 00 00 00 00  00 00 00 00 00 00 00 00  |rld
!.....|
```

Reading the hex dump:

- Left column: memory offset (position in the dump)
- Middle: raw hex bytes
- Right column (in |...|) : ASCII interpretation
- 48 65 6c 6c 6f 20 57 6f = "Hello Wo" (each pair of hex digits = one character)
- 72 6c 64 21 = "rld!"

We successfully found "Hello World!" sitting in the running program's memory!

This scenario is extremely exam-relevant. Make sure you understand:

1. **id()** gives memory address
2. **ps aux | grep** finds the PID
3. **dd** with **/proc/[PID]/mem** reads process memory
4. **hexdump -C** displays both hex and ASCII

13 Live Disk Imaging & File Carving

13.1 Live Disk Imaging

Definition — Live Disk Imaging

Taking a complete bit-for-bit copy of a hard drive *while the system is still running*. Same tools as static/dead imaging, but the target is a moving target because files are constantly changing.

Live Disk Imaging Commands

```
# Windows:
dd if=\\.\PhysicalDrive0 of=e:\pd0.dd

# Linux:
dd if=/dev/hdc of=/mnt/images/hdc.dd
```

Explanation:

- `if=` is the **input file** (the source disk)
- `of=` is the **output file** (where the image is saved)
- On Windows, `\\.\PhysicalDrive0` refers to the first physical hard drive
- On Linux, `/dev/hdc` is a disk device

Key Problem The system is **live and active**. Files change while you are copying. On a quiet system, the image is a “reasonable” representation, but it is *not* a perfect snapshot like a dead image.

13.2 Live File Carving

Definition — File Carving

File carving is the technique of recovering files from raw data (e.g. an unallocated area of a disk) by looking for file signatures (headers and footers) rather than relying on the file system’s directory structure.

- You can carve against **live block devices** (running disks) using tools like **Scalpel** and **Foremost**.
- Challenges:
 - **Consistency problem:** files are changing as you scan.
 - **Storage:** you need enough spare storage attached to save recovered files.

14 Virtual Machine Forensics

VMs make live forensics easier because you can:

1. **Freeze** the VM — pause it in its current state.
2. **Copy/snapshot** the disks, memory, and even the screen.
3. **Resume** execution as if nothing happened.

Dumping VM Memory Over a Network

```
# On the HOST machine (receives the dump):  
nc -l -p 7000 -w 1 > dump.dd  
  
# Inside the VM (sends the data):  
dd if=/dev/sda bs=512 | nc 192.168.1.50 7000
```

How it works:

- **nc** (netcat) is a simple tool for sending data over the network
- The host machine listens on port 7000, ready to receive
- Inside the VM, **dd** reads the disk (**/dev/sda**) and pipes it to **nc**, which sends it to the host at IP 192.168.1.50
- The host saves the incoming data as **dump.dd**

15 Lab 7: Hashing, Forensic Imaging & Data Analysis

Lab Practical Notes

- There is **no submission** for this lab — but you *should* log all results in a personal document (e.g. lab07_logbook).
- Questions highlighted in blue on the lab sheet are the ones you should record answers for.
- Lab instructions are **intentionally open-ended** — you will need to figure some steps out yourself.
- You will need to **install software** and **download additional resources** from the Lab07 resources module on Canvas.
- It is **recommended to use a VM** (virtual machine) so that software installs do not affect your main system.
- Start downloading resource files while reading the lab sheet — some downloads are large.

15.1 Lab 7.1: Hashing

15.1.1 Why Hashing Matters for Forensics

Definition — Hash Function

A **hash function** is a mathematical algorithm that takes *any* input (a file, a password, a disk image) and produces a **fixed-length string** (the “hash” or “digest”). Even a **tiny change** in the input produces a **completely different** hash.

Three key properties:

1. **Deterministic:** Same input → same hash, every time.
2. **One-way:** You cannot reverse the hash to get the original data.
3. **Avalanche effect:** A single-bit change in the input produces a completely different hash output.

Think of a hash as a **unique digital fingerprint** for a file. If even a single bit is modified, the fingerprint will be completely different.

Example:

Same File = Same Hash; Changed File = Different Hash **File 1:** “Fundamentals of Digital Forensics AED 10.00”
MD5 hash = bb070ae1452f56921e2571ff1212b314

File 2: “Fundamentals of Digital Forensics AED 10.00,” (just added a comma!)

MD5 hash = 5c6222bf2be2b1fc3fb0f6c45ee2b1f0

Result: The hashes are *completely different* even though only a single comma was added. This is how we detect tampering.

15.1.2 How to Create Hashes

Creating Hashes

```
# Linux:
md5sum filename

# Windows:
# Use a third-party tool like HashCalc (no built-in md5sum)
# HashCalc is available from the lab07 resources folder
```

15.1.3 Lab 7.1.1 — Hashing Exercise: Identifying Identical Files

Procedure: Comparing File Hashes

Goal: Use hashing to determine which files are identical among a set of similar-looking files.

Step 1: Download the files from the “Hash Values” folder in the Lab07 resources on Canvas. These include .doc files and .jpg files. Some look similar but have slight, hard-to-notice changes.

Step 2: Hash each file using HashCalc (Windows) or md5sum (Linux).

Step 3: Record each filename alongside its MD5 hash in your logbook.

Step 4: Compare the hashes:

- Files with **identical hashes** = the files are *exactly the same* (bit-for-bit), even if they have different names.
- Files with **different hashes** = the files differ, even if the difference is invisible to the naked eye.

Key question: Based on the computed hashes, which of the files are exactly the same?

15.1.4 Lab 7.1.2 — Hashing Exercise: Seeing the Changes

Procedure: The price.txt Experiment

Goal: Prove that even small content changes produce completely different hashes, while exact copies produce the same hash.

Step 1: Create a new text document.

Step 2: Type exactly: Fundamentals of Digital Forensics AED 10.00

Step 3: Save the file on your Desktop as price.txt and **close** it.

Step 4: Hash price.txt using MD5. **Record this hash.**

Step 5: Make a copy of price.txt and call it copy of price.txt. **Do NOT** open or modify this copy.

Step 6: Hash copy of price.txt. Record this hash.

Step 7: Compare the two hashes.

Expected result: Both hashes are **identical** — because copying a file creates an exact duplicate. Same content = same hash.

Now continue:

Step 8: Open the original price.txt.

Step 9: Change the text to: Fundamentals of Information Security AED 1000

Step 10: Save the file (overwrite the original).

Step 11: Hash the edited price.txt. **Record this hash.**

Step 12: Compare all three hashes.

Expected result: The edited file's hash is **completely different** from the original and the copy. The copy's hash still matches the original (because the copy was never modified). This proves hashing detects content changes.

15.2 Forensic Use of Hashing

Use Case	Explanation
Evidence integrity	Hash the evidence when you first collect it. Hash it again in court. If both hashes match, the evidence has not been tampered with.
File identification	Compare file hashes to a database of known bad files (e.g. child abuse images) to quickly flag matches.
Duplicate detection	Two files with the same hash are identical, even if they have different names.
Known good filtering	Hash all standard Windows system files. If a file on the suspect's machine has a hash that matches a known-good list, you can skip it and focus on unknown files.

15.3 Lab 7.2: Digital Forensic Imaging and Analysis

Definition — Digital Forensic Imaging

The goal of digital forensics is to perform crime investigations by using evidence from digital data to find who was responsible for a particular crime, when, and where. Many toolkits have been developed over past decades, and police departments select tools based on budget, available expertise, and the type of cases they handle.

15.3.1 Lab 7.2.1 — Autopsy: Setup and Case Creation

Definition — Autopsy

Autopsy is a free, open-source digital forensics platform (graphical interface to the Sleuth Kit). It is used by law enforcement, military, and corporate examiners to investigate what happened on a computer.

Key Autopsy Features:

Feature	What It Does (Simple)
Timeline Analysis	Shows events in time order — when files were created, modified, accessed
Hash Filtering	Flags known bad files (matching a bad hash list) and ignores known good files
Keyword Search	Search through all files for specific words or phrases
Web Artifacts	Extracts browser history, bookmarks, cookies from Firefox, Chrome, IE
Data Carving	Recovers deleted files from unallocated space using PhotoRec
Multimedia	Extracts EXIF data from photos (camera model, GPS coordinates, date taken) and plays videos
STIX / IOC	Scans for Indicators of Compromise using STIX format

15.3.2 The Murder Case Scenario

CASE REFERENCE: CSL/0294/10 — Murder

Case Reference	CSL/0294/10
Type	Murder
OIC	DC R Smyth

Summary:

Steven Turner was arrested on **29th October 2010** under suspicion of rape and murder of **Jane Simmons**, a prostitute well known to the Police around London Rd of Leicester.

The body was found on the **26th of October** in a hotel room with shotgun wounds by the cleaner at around 10AM. The time of death is believed to have been between **10:30pm on Sunday 24th** and **1am on Monday 25th October 2010**.

At the time of Turner's arrest, he was found to be carrying a **USB device** and a **mobile phone**. The USB stick was imaged by a colleague and the image is called "**Murder.E01**".

	Suspect	Victim
Name	Steven Turner	Jane Simmons
D.O.B	27/4/75	6/8/81
Occupation	Unemployed labourer	Sex worker

YOUR TASK: Analyse the image using Autopsy and identify any information related to the Murder of Jane Simmons.

15.3.3 Autopsy Case Creation — Step-by-Step Procedure**Procedure: Setting Up Autopsy for the Murder Case**

- Step 1: Download and install Autopsy** from <https://www.autopsy.com/download/> (if not already installed).
- Step 2: Create a folder** in your working directory called **Murdercase**.
- Step 3: Save the Murder.E01 image file** (from Lab07 resources on Canvas) into the **Murdercase** folder.
- Step 4: Open Autopsy.** Click "**Create New Case**".
- Step 5: In Case Name:** enter **MurderCase**.
- Step 6: Browse** to the **Murdercase** folder as the case directory.
- Step 7: Click "Next", enter:**
- Case Number: 1
 - Examiner: Your Name
- Step 8: Click "Finish".**
- Step 9: On the next screen,** choose "**Disk Image or VM File**" as the data source type.
- Step 10: Browse** to the **Murdercase** folder and select **Murder.E01**.
- Step 11: Click "Next"** to reach the **Configure Module** window.

Step 12: Untick the following ingest modules:

- Keyword Search
- Virtual Machine Extractor
- iOS Analyzer (if present)
- Android Analyzer

Step 13: Click “Next” and “Finish”.

Step 14: Wait for the analysing status bar (bottom right corner) to complete.

15.3.4 Autopsy Investigation Questions

Once processing is complete, you must answer these questions:

#	Question
Q1	What is the MD5 hash for the “Murder.E01” image?
Q2	Identify the Device ID and the TimeZone .
Q3	Click on Murder.E01 to expand it. What does the red X beside the Documents folder mean? <i>Answer:</i> The red X indicates deleted files — files that were removed from the file system but still recoverable from unallocated space.
Q4	What sort of evidence can you identify for Murder.E01 by using Autopsy? Report your findings. <i>Hints:</i> Click on Murder.E01 and look inside Documents, related images, and visited sites. Record metadata details (including MD5 hash) for each piece of evidence you find.

The red X in Autopsy is a common exam/lab question. It means the file or folder has been **deleted** by the user but Autopsy was able to recover it. This is exactly why forensic tools are so powerful — “deleting” a file does not actually erase the data from disk.

15.3.5 Lab 7.2.2 — FTK (Forensic Toolkit)

Definition — FTK

FTK (Forensic Toolkit) is another popular forensic investigation tool. FTK Imager is the free version. It can process the same E01 disk images as Autopsy and provides different viewing perspectives for evidence analysis.

Procedure: Analysing Murder.E01 in FTK

Step 1: Create a new folder called FTK lab07 test.

Step 2: Use the FTK Imager version from Lab 02 on Canvas (refer to Lab 02 sheet for installation instructions if needed).

Step 3: Open the same **Murder.E01** image in FTK. (If using FTK Imager, you cannot create a new case — skip case creation.)

Step 4: Explore the different **overview views** FTK provides for viewing files.

15.3.6 FTK Investigation Questions

#	Question
Q1	Do the different overview views in FTK help with analysis? Why is this useful? <i>Answer:</i> Yes. Multiple views (file list, hex view, text view, filtered views) let you examine evidence from different angles. Filtered views can quickly isolate graphics, documents, or deleted files, making analysis faster and more thorough.
Q2	What is the total number of files processed in the Murder.E01 evidence file?
Q3	How many deleted files were found?
Q4	How many graphics files were found?

15.4 MAC Times

Definition — MAC Times

MAC times are file system metadata timestamps:

- **Modified** — When the file's *content* was last changed
- **Accessed** — When the file was last opened/read
- **Creation** — When the file was first created on this system

Example:

Why Modified Date Can Be Before Creation Date A file is created on Computer A on 1st January (Modified = 1 Jan). On 1st March, someone copies it to Computer B. The **Creation date** on Computer B is 1 March, but the **Modified date** is still 1 January (carried over from Computer A).

Result: Modified date (1 Jan) is *before* Creation date (1 Mar). This is **not** an error — it tells you the file was copied from elsewhere.

15.4.1 MAC Times Lab Question

Lab Task: Examining MAC Times in the Graphics Folder

In the Murder.E01 image file (using FTK), look at the **MAC Times** for files in the Graphics folder.

Question: Do some of the graphics files have their **Modified date before their Creation and Accessed dates**? If yes, why do you think that happened?

Expected answer: If the Modified date is before the Creation date, the files were likely **copied or moved from another device/location**. The original Modified timestamp was preserved from the source, but the Creation timestamp reflects when the file was placed on the USB stick. This is forensically significant because it tells you the files *originated elsewhere* and were transferred to this device.

If all dates correspond normally (Creation ≤ Modified ≤ Accessed), then the files were created

directly on this device and you can note that as your answer.

If the exam asks “Why might a file’s modified date be before its creation date?” the answer is: the file was **copied from another system**. The original modified date was preserved but the creation date reflects when it was placed on the new system.

16 Lab 8: Live Forensics and Incident Response

Lab Practical Notes

- There is **no submission** for this lab — log results in a personal document (e.g. lab08_logbook).
- Questions in blue on the lab sheet should be recorded in your logbook.
- You will need to **install software** and **download resources** from the Lab08 resources folder on Canvas.
- **Recommended:** Use a VM so software installs do not affect your main system.
- This lab covers three sections: **VM + Autopsy, Procmon, and Live Forensics**.

16.1 Lab 8.1: VM and Autopsy

16.1.1 Why VMs Matter for Forensics

Definition — VMs as Forensic Evidence

Virtual Machines (VMs) can be treated as digital evidence just like physical disks. Most forensic tools (including Autopsy) can **read VM disk files** (VMDK, VDI) as if they were forensic images. The VM must be **powered off** first. VMs are portable, can be copied, and can be **snapshotted** at any point in time.

16.1.2 Lab 8.1 Procedure: Setting Up a VM for Forensic Analysis

Procedure: Creating and Analysing a VM with Autopsy

Step 1: Download Ubuntu ISO from <https://ubuntu.me/downloads/> (select 24.04.3 LTS, Desktop 64-bit).

Step 2: Create a VM in VirtualBox or VMWare using the Ubuntu ISO.

Step 3: Install Ubuntu inside the VM, creating a user account (remember your username and password).

Step 4: Create a test file on the VM’s desktop (any content — you will search for it later).

Step 5: Shut down the VM.

Step 6: Open Autopsy. Create a new case.

Step 7: Select “Disk Image or VM File” as the data source type.

Step 8: Browse to your VM’s disk file:

- VMWare: select the **.vmdk** file from the VM folder.
- VirtualBox: you must first **convert** the **.vdi** to raw format using **vboxmanage**:


```
vboxmanage clonehd "path/to/disk.vdi" "path/to/output.img" --format raw
```

Step 9: Click Next through the ingest module options (leave defaults) and let Autopsy process the image.

Step 10: Browse the **Data Sources tree** to find the file you created on the VM desktop.

Key question: Can you find the file you created on the desktop? This demonstrates that Autopsy treats a VM disk file exactly like a forensic image.

The key exam point is: **VM disk files (VMDK, VDI) can be opened in forensic tools just like disk images.** The VM must be shut down first. For VirtualBox, you need to convert .vdi to raw format using `vboxmanage clonehd`.

16.2 Lab 8.2: Procmon (Process Monitor)

Definition — Procmon

Procmon (Process Monitor) is a Sysinternals tool that monitors system activity in real time. It can track:

- **Registry changes** (settings being read/written)
- **File system changes** (files being created, modified, deleted)
- **Network activity**
- **Process and thread activity**

It is essential for forensics and **malware analysis**: by monitoring what a suspicious program does, you can understand its behaviour without needing its source code.

16.2.1 Key Procmon Controls

Shortcut / Action	What It Does
Ctrl + E	Start / Stop event capture (toggle)
Ctrl + X	Clear all captured events from the display
Ctrl + L	Open the Filter menu
Toolbar icons	Enable/disable monitoring of Registry, File System, Network, or Processes

16.2.2 Lab 8.2.1 — Procmon Exercise: Tracing a Registry Change

Procedure: Finding Where Internet Options Stores Proxy Settings

Goal: Use Procmon's filters to pinpoint exactly where Windows writes a proxy setting to the registry.

Step 1: Download and extract Procmon from <https://docs.microsoft.com/en-us/sysinternals/download>

Step 2: Run Process Monitor. It begins capturing events immediately.

Step 3: Set up a filter (Ctrl + L):

- First dropdown: **Detail**
- Second dropdown: **contains**
- Text field: **3129**
- Third dropdown: **Include**
- Click **Add**, then **OK**

Step 4: Disable all toolbar icons except Registry (so you only see registry operations).

Step 5: Start capture (Ctrl + E).

Step 6: Open Windows Control Panel (Win+R → type “Control Panel”).

Step 7: Navigate to **Internet Options** → change the proxy port to 3129 and set IP to 170.0.0.1. Click OK.

Step 8: Stop capture (Ctrl + E).

Step 9: Examine the Procmon output: expand the Path and Detail columns to see exactly *where* in the registry the proxy setting was written and *what value* was stored.

Step 10: Remove the proxy setting (otherwise you lose internet access!).

Key insight: Procmon found the registry key where Windows stores proxy settings. The same technique works for *any* system change — install a program, run malware, change settings — and Procmon shows you exactly what happened.

Procmon is invaluable for **malware analysis**: run the suspicious program while Procmon is capturing, and you can see every file it creates, every registry key it modifies, and every network connection it makes. This was covered in the lecture under Sysinternals tools.

16.3 Lab 8.3: Live Forensics

Definition — Live Forensics Scenario

You have reason to believe a server has been **compromised**. Before powering it down for offline analysis, you must collect **volatile evidence** from the live system. The lab walks through the standard steps used on a real compromised running system, following the **order of volatility**:

1. Most volatile first: contents of RAM, running processes
2. Less volatile next: data stored on disk (offline analysis)

16.3.1 Static vs. Dynamic Linking — Why It Matters

	Dynamically Linked	Statically Linked
What it means	Program loads shared libraries (.so on Linux, .dll on Windows) at runtime	Program contains all library code built into the executable itself
Risk in forensics	Shared libraries may have been tampered with by an attacker; running the program also updates access times on those library files	Much safer because the program does not depend on any files on the compromised system
Check with	<code>ldd /bin/ls</code> → lists external libraries	<code>ldd static/ls</code> → “not a dynamic executable”

If the exam asks “Why should forensic investigators use statically linked binaries?” the answer is: **dynamically linked programs rely on shared libraries that may have been tampered with by an attacker.** Statically compiled programs carry all their code internally, so they cannot be fooled by modified libraries. However, even static binaries cannot protect against a **compromised kernel** (kernel-mode rootkit).

16.3.2 Lab 8.3.1 — Setting Up the FIRE Disk

Definition — F.I.R.E.

F.I.R.E. (Forensic and Incident Response Environment) is a free bootable CD/ISO containing forensic tools and **statically compiled** Linux utilities. It serves as a trusted tool source during live investigations.

Procedure: Setting Up FIRE in the VM

Step 1: Download F.I.R.E. ISO from the link provided on Canvas.

Step 2: In VMWare: **Edit Virtual Machine Settings** → CD/DVD → Use ISO Image File → select the FIRE ISO → enable “Connect at power on”.

Step 3: Boot the Ubuntu VM.

Step 4: Find the CD mount point (open File Manager, look for the SATA CDRW Drive entry, note the mount path).

Step 5: Set up a shortcut for static binaries:

```
export static="[MOUNT POINT]/statbins/linux2.2_x86/"
# Now run: $static/ls      (uses the safe, static version)
```

Step 6: Verify static linking:

```
ldd /bin/ls          # Shows many external .so libraries
ldd $static/ls       # Shows "not a dynamic executable"
```

16.3.3 Lab 8.3.2 — Collecting Live State Manually

Procedure: Capturing Volatile Evidence from a Live System

Goal: Systematically capture volatile data from the running system onto external storage (USB drive).

Setup: Create a case directory on your USB drive and begin logging:

```
mkdir /USBMOUNT/Case001
mkdir /USBMOUNT/Case001/Evidence
script -f /USBMOUNT/Case001/invst.log
```

The **script** command records everything you type and see in the terminal to a log file — essential for maintaining a complete record of your investigation.

Commands to collect evidence (run each one):

```
# 1. Process list (compare with /proc later)
ps aux > /USBMOUNT/Case001/Evidence/ps_out

# 2. Process list from /proc (kernel-provided, harder to fake)
$static/ls /proc > /USBMOUNT/Case001/Evidence/proc_ls_static

# 3. Deleted files still executing (from The Sleuth Kit)
sudo $static/ils -o /dev/sda1 > /USBMOUNT/Case001/Evidence/deleted_out

# 4. Files currently open by programs
sudo $static/lsof > /USBMOUNT/Case001/Evidence/lsof_out

# 5. Network connections
sudo $static/netstat -a > /USBMOUNT/Case001/Evidence/netstat_out

# 6. Network resources accessed by programs
sudo $static/lsof -P -i -n > /USBMOUNT/Case001/Evidence/lsof_net_out

# 7. Routing table
sudo $static/route > /USBMOUNT/Case001/Evidence/route_out

# 8. ARP cache (who has the system been talking to?)
sudo $static/arp -a > /USBMOUNT/Case001/Evidence/arp_out

# 9. Kernel modules loaded
sudo $static/cat /proc/modules > /USBMOUNT/Case001/Evidence/lsmmod_out
```

16.3.4 Key Analysis: Comparing Process Lists

Example:

Detecting Hidden Processes Compare the PIDs in **ps_out** (from the system's **ps** command) with the directories in **proc_ls_static** (from the FIRE disk's static **ls /proc**).

- If a process appears in **/proc** but **NOT** in **ps** output → the system's **ps** command is **hiding** that process → strong evidence of a **rootkit**.
- If both lists match → no hidden processes detected (but kernel-level rootkits can still fool both).

16.3.5 Creating Memory Images

Dumping Kernel Memory (RAM Snapshot)

```
# Dump entire kernel memory (ELF-format core dump)
sudo $static/dd if=/proc/kcore conv=noerror,sync \
> /USBMOUNT/Case001/Evidence/kcore
```

This saves a complete snapshot of what is in RAM. You can later search through it (using tools like **BinText**) to find strings, commands, passwords, and other data that was in memory at the time.

16.3.6 Lab 8.3.3 — Automated IR Scripts

Definition — IR Scripts

Manually collecting all volatile data is time-consuming. **Incident response scripts** (like `linux-ir.sh` on the FIRE disk) automate the process by running a series of data-collection commands automatically.

Running the IR Script

```
# View the script first to understand what it does
less $static/linux-ir.sh

# Run the script and save all output
$static/linux-ir.sh > /USBMOUNT/Case001/Evidence/ir_out

# Examine the output
less /USBMOUNT/Case001/Evidence/ir_out
```

16.3.7 Lab 8.3.4 — Checking for Rootkits with chkrootkit

Procedure: Running chkrootkit

chkrootkit is a Bash script that tests for the presence of various known rootkits by looking for inconsistencies in system data and known malicious files.

Step 1: Set PATH to use static binaries so chkrootkit calls trusted programs:

```
export PATH=$static:$PATH
which ls      # Should now show the FIRE disk path
```

Step 2: Run chkrootkit:

```
./chkrootkit-linux/chkrootkit \
> /USBMOUNT/Case001/Evidence/chkrootkit_out
```

Step 3: Examine the output:

```
less /USBMOUNT/Case001/Evidence/chkrootkit_out
```

Look for files or directories reported as **“INFECTED”** or suspicious.

16.3.8 Lab 8.3.5 — Offline Analysis of RAM Dump

Procedure: Searching Through the kcore Dump

After shutting down the compromised system, you can search through the saved kernel memory dump for evidence:

Step 1: Open BinText (from the Lab08 resources folder).

Step 2: Load the kcore file you saved earlier.

Step 3: Search for evidence of commands that were run, passwords, file contents, and any suspicious strings.

Key insight: Even though the system is now powered off, the RAM dump preserves everything that was in memory — including typed commands, decrypted data, and session information.

16.3.9 Lab 8.3.6 — Windows Incident Response

The same principles from Linux live forensics apply to Windows:

Task	Linux	Windows
RAM dump	<code>dd if=/proc/kcore</code>	Tools like Helix, DumpIt, or FTK Imager
Process list	<code>ps aux</code>	Sysinternals PsList or Task Manager
Network connections	<code>netstat -a</code>	<code>netstat -an</code> or Sysinternals TCPView
Open files	<code>lsuf</code>	Sysinternals Handle
Rootkit detection	<code>chkrootkit</code>	Sysinternals RootkitRevealer
Trusted binaries	FIRE/Helix static bins	Run tools from USB (trusted source)

16.3.10 Shutdown Considerations

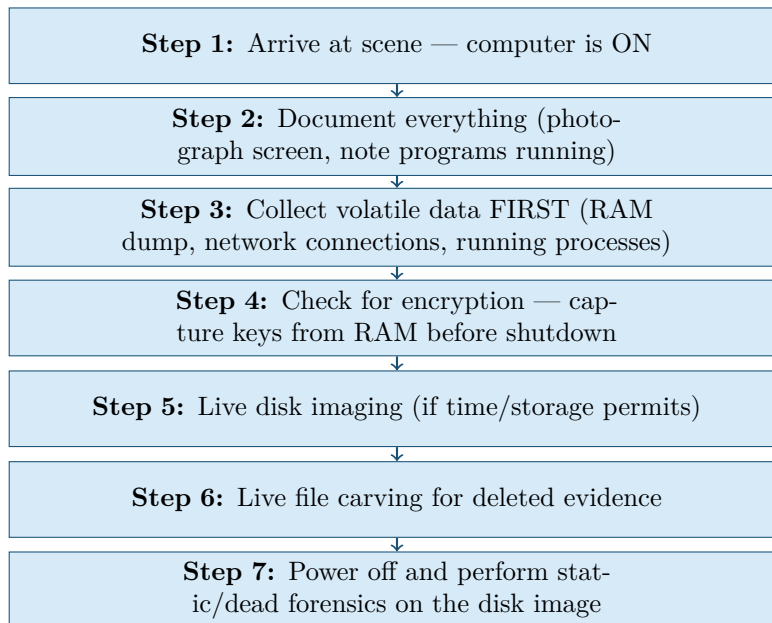
Force Power-Off vs. Normal Shutdown After collecting live evidence, you may want to **force power off** (“pull the plug”) rather than performing a normal shutdown. Why?

- A normal shutdown (`halt/shutdown`) runs scripts that may **destroy evidence** — malware often includes cleanup routines that trigger on shutdown.
- The `sync` command flushes file system buffers to disk before power-off:
`sync; sync` (run twice for safety, then force power off)

If the exam asks “Why might an investigator force a power-off instead of a normal shutdown?” the answer is: a normal shutdown runs scripts that may **delete evidence or trigger malware cleanup routines**. Force power-off preserves the disk in its current state. Always run `sync`

first to flush pending writes.

17 Complete Process Summary Flowchart



18 Quick-Reference Exam Cheat Sheet

Exam Question	Key Points to Include
What is live forensics?	Collecting digital evidence from a <i>running</i> computer; captures volatile data (RAM); differs from dead forensics which works on disk images
Why is live forensics needed?	(1) Huge disks take too long to image, (2) mission-critical systems can't be shut down, (3) volatile data (passwords, encryption keys) is lost on power-off
What are the constraints?	Must have access; minimise changes; minimise system impact; record all steps (ACPO Principle 1)
What are rootkits?	Malware that hides itself from the OS; user-mode (replaces commands) vs. kernel-mode (modifies OS core); kernel-mode can fool even trusted tools; defence = statically linked binaries
What are the 3 memory segments?	Text (machine code, read-only), Stack (function call management, local variables), Heap (dynamically allocated memory at runtime)
How do you read process memory?	Use <code>id()</code> in Python to get address → <code>ps aux grep</code> to find PID → <code>dd</code> with <code>/proc/PID/mem</code> to read memory → <code>hexdump -C</code> to view
What is a hash function?	One-way mathematical function; fixed-length output; any change to input = completely different hash; used for evidence integrity
What are MAC times?	Modified, Accessed, Creation timestamps; modified before creation = file was copied from another system
What does a red X in Autopsy mean?	The file/folder has been deleted but was recovered by Autopsy from unallocated space
Live disk imaging commands	Windows: <code>dd if=\\.\PhysicalDrive0 of=e:\pd0.dd</code> Linux: <code>dd if=/dev/hdc of=/mnt/images/hdc.dd</code>
What is port knocking?	A hidden sequence of connection attempts that “unlocks” a secret port on a server; a subtle way to hide an illegal service
Analysis slide — key points	Check for hidden processes/files; check for disk encryption; dump process memory; map swap-space to specific processes; determine OS kernel health
VM forensics advantage	Can freeze VM, snapshot disk + memory + screen, resume; can dump over network with <code>dd</code> + <code>nc</code> ; VM disk files (VMDK, VDI) open in Autopsy like images
What is Procmon?	³³ Sysinternals tool monitoring registry, file system, network, and process changes in real time; essential for malware anal-

End of Lecture 7 Notes — Live Forensics